**Abstract**

This paper presents a comprehensive, end-to-end machine learning pipeline applied to the E-Commerce Fraud Detection Dataset. Our pipeline encompasses: complete data preprocessing including timestamp decomposition, IQR-based outlier capping, and country-mismatch feature engineering; multi-dimensional exploratory data analysis revealing temporal, geographic, and merchant-category fraud patterns; K-Nearest Neighbour analysis under multiple distance metrics and scaling strategies; feature selection via correlation analysis, univariate scoring (SelectKBest), Recursive Feature Elimination (RFE), Lasso regression, and Random Forest importance; three class-imbalance handling strategies ; one-hot encoding of categorical features; domain-driven feature construction (security score, geo-risk, user activity score, night transaction flag); Logistic Regression with threshold optimisation and a full classification report; Decision Tree overfitting analysis at varying depths; SGDClassifier training with four regularisation strategies (L1, L2, Elastic Net, and early stopping) using polynomial features; a from-scratch NumPy neural network with He-initialised weights, forward propagation, binary cross-entropy loss, ReLU and sigmoid activations, backpropagation, and gradient-descent parameter updates, cross-validated against equivalent TensorFlow/Keras and PyTorch implementations; and Random Forest hyperparameter tuning via both RandomizedSearchCV and the Bayesian Optuna framework. The best model, a Random Forest tuned by RandomizedSearchCV, achieves 99.27% accuracy, F1 = 0.7768, and AUC-ROC = 0.9464. We provide a thorough analysis of why the custom neural network failed to detect fraud , explain the performance differences between regularisation strategies, and justify each design decision throughout the pipeline.

**Keywords:** E-Commerce, Fraud Detection, Class Imbalance, SMOTE, Feature Selection, Neural Networks, Backpropagation, Random Forest, Regularisation, Hyperparameter Optimisation, Optuna

# 1 Introduction

The explosive growth of online commerce has created a parallel expansion in digital payment fraud. Fraudulent transactions impose direct financial losses on merchants and cardholders, erode consumer confidence, and increase operational costs through chargeback handling and dispute resolution. The challenge is compounded by the inherent rarity of fraud: in real transaction streams, fraudulent events typically represent fewer than 2% of all records, making standard classifiers that optimise for accuracy inadequate without specialised handling.

Traditional rule-based fraud detection systems assign scores based on predefined thresholds. While interpretable, they cannot generalise beyond the rules they encode and require constant manual updating as fraud patterns evolve. Machine learning offers an adaptive alternative: models can discover complex, non-linear interactions among transaction features automatically, learning patterns that human analysts might overlook.

This report documents a complete, reproducible machine learning pipeline applied to the publicly available E-Commerce Fraud Detection Dataset. Rather than jumping straight to a final model, the pipeline systematically investigates every stage of the data science workflow: what the data looks like, what the fraud patterns are, how features should be prepared, which features matter most, how to handle the severe class imbalance, and what happens when we apply progressively more sophisticated models. This methodical approach makes the final results interpretable and the design decisions justifiable.

# 2 Dataset Description

## 2.1 Shape and Schema

The raw dataset contains 299,695 rows and 17 columns. Table 1 summarises the feature schema.

**Table 1** Dataset schema: column names, types, and non-null counts

| # | Column | Dtype | Non-Null Count |
|---|---|---|---|
| 0 | `transaction_id` | int64 | 299,695 |
| 1 | `user_id` | int64 | 299,695 |
| 2 | `account_age_days` | int64 | 299,695 |
| 3 | `total_transactions_user` | int64 | 299,695 |
| 4 | `avg_amount_user` | float64 | 299,695 |
| 5 | `amount` | float64 | 299,695 |
| 6 | `country` | object | 299,695 |
| 7 | `bin_country` | object | 299,695 |
| 8 | `channel` | object | 299,695 |
| 9 | `merchant_category` | object | 299,695 |
| 10 | `promo_used` | int64 | 299,695 |
| 11 | `avs_match` | int64 | 299,695 |
| 12 | `cvv_result` | int64 | 299,695 |
| 13 | `three_ds_flag` | int64 | 299,695 |
| 14 | `shipping_distance_km` | float64 | 299,695 |
| 15 | `is_fraud` | int64 | 299,695 |
| 16 | `transaction_time` | object | 299,695 |

### 2.2 Feature Semantics

The features can be grouped into four categories. *User-level attributes* include `account_age_days` (how long the account has existed), `total_transactions_user` (total historical transaction count for that user), and `avg_amount_user` (the user's historical average transaction value). *Transaction-level attributes* include `amount` (the current transaction value), `merchant_category` (sector of the merchant), `channel` (payment channel: web or app), `country` (transaction origin country), `bin_country` (card-issuing country inferred from the BIN), `shipping_distance_km` (kilometres to shipping destination), and `promo_used` (whether a promotional code was applied). *Security verification flags* are `avs_match` (Address Verification System pass), `cvv_result` (CVV code match), and `three_ds_flag` (3-D Secure authentication). Finally, `transaction_time` is a datetime string and `is_fraud` is the binary target label.

### 2.3 Data Quality

Running `df.isna().sum() / len(df)` confirms a missing-value rate of exactly 0.0 for every column, and `df.duplicated().sum()` returns 0, confirming no exact duplicate rows. This is an unusually clean dataset, requiring no imputation or deduplication. The first five rows show a sample transaction from France (`country` = FR, `bin_country` = FR) in the `travel` merchant category through the `web` channel, confirming the feature semantics above.

- **Data availability**: https://www.kaggle.com/datasets/umuttuygurr/e-commerce-fraud-detection-dataset

## 3 Data Preprocessing

### 3.1 Identifier Removal

The columns `transaction_id` and `user_id` were dropped first. These are surrogate keys that carry no predictive signal and would cause data leakage or spurious correlations if retained.

### 3.2 Timestamp Decomposition

The `transaction_time` column was parsed as a `datetime` object and decomposed into four cyclical components: `hour` (0–23), `day` (1–31), `month` (1–12), and `weekday` (0=Monday, 6=Sunday). The original string column was then dropped. This decomposition allows tree-based and linear models to exploit temporal patterns (for instance, fraud may concentrate in night hours or specific months) without requiring the model to parse raw timestamps.

### 3.3 Descriptive Statistics

Table 2 shows key descriptive statistics from `df.describe().T`, revealing important properties of the continuous features.

**Table 2** Descriptive statistics for numerical features

| Feature | Count | Mean | Std | Min | 25% | 50% | 75% | Max |
|---|---|---|---|---|---|---|---|---|
| account_age_days | 299,695 | 973.40 | 525.24 | 1.00 | 516.00 | 975.00 | 1425.00 | 1890.00 |
| total_transactions_user | 299,695 | 50.67 | 5.98 | 40.00 | 46.00 | 51.00 | 56.00 | 60.00 |
| avg_amount_user | 299,695 | 148.14 | 200.36 | 3.52 | 46.19 | 90.13 | 173.45 | 4565.29 |
| amount | 299,695 | 177.17 | 306.93 | 1.00 | 42.10 | 89.99 | 191.11 | 16994.74 |
| promo_used | 299,695 | 0.154 | 0.361 | 0.00 | 0.00 | 0.00 | 0.00 | 1.00 |
| avs_match | 299,695 | 0.838 | 0.368 | 0.00 | 1.00 | 1.00 | 1.00 | 1.00 |
| cvv_result | 299,695 | 0.872 | 0.334 | 0.00 | 1.00 | 1.00 | 1.00 | 1.00 |
| three_ds_flag | 299,695 | 0.785 | 0.411 | 0.00 | 1.00 | 1.00 | 1.00 | 1.00 |
| shipping_distance_km | 299,695 | 357.05 | 427.67 | 0.00 | 136.60 | 273.02 | 409.18 | 3748.56 |
| is_fraud | 299,695 | 0.022 | 0.147 | 0.00 | 0.00 | 0.00 | 0.00 | 1.00 |
| hour | 299,695 | 11.51 | 6.93 | 0.00 | 5.00 | 12.00 | 18.00 | 23.00 |
| day | 299,695 | 15.75 | 8.81 | 1.00 | 8.00 | 16.00 | 23.00 | 31.00 |
| month | 299,695 | 5.52 | 2.88 | 1.00 | 3.00 | 6.00 | 8.00 | 10.00 |
| weekday | 299,695 | 2.97 | 2.00 | 0.00 | 1.00 | 3.00 | 5.00 | 6.00 |

Binary variables (promo_used, avs_match, cvv_result, three_ds_flag, is_fraud) have means representing proportions. The fraud rate (0.022) confirms strong class imbalance.

## 3.4 Outlier Detection and Capping

Outliers in continuous features were detected using the Interquartile Range (IQR) method. For each feature, the lower and upper bounds are:

$$L = Q_1 - 1.5 \cdot (Q_3 - Q_1) \tag{1}$$

$$U = Q_3 + 1.5 \cdot (Q_3 - Q_1) \tag{2}$$

Any value outside $[L, U]$ is classified as an outlier. The detection was applied to all numeric columns. Table 3 shows the results before capping.

**Table 3** Outlier summary for numeric features before IQR capping

| Feature | Q1 | Q3 | Lower Bound | Upper Bound | Outlier Count |
|---|---|---|---|---|---|
| avg_amount_user | 46.19 | 173.45 | –144.70 | 364.34 | 23,890 |
| amount | 42.10 | 191.11 | –181.42 | 414.63 | 26,711 |
| promo_used | 0.00 | 0.00 | 0.00 | 0.00 | 46,045 |
| avs_match | 1.00 | 1.00 | 1.00 | 1.00 | 48,551 |
| cvv_result | 1.00 | 1.00 | 1.00 | 1.00 | 38,328 |
| three_ds_flag | 1.00 | 1.00 | 1.00 | 1.00 | 64,558 |
| shipping_distance_km | 136.60 | 409.18 | –272.27 | 818.05 | 20,312 |
| is_fraud | 0.00 | 0.00 | 0.00 | 0.00 | 6,612 |

Note: The large "outlier" counts for binary features (promo_used, avs_match, cvv_result, three_ds_flag) occur because when both $Q_1 = Q_3 = 1$, the IQR is zero and any value deviating from 1 (i.e., the value 0) is flagged as an outlier. This is a known artefact of the IQR method on binary variables and does not represent genuine outliers.

For the five designated continuous features (`account_age_days`, `total_transactions_user`, `avg_amount_user`, `amount`, `shipping_distance_km`), values outside the IQR bounds were *capped* (Winsorised) using `pandas.clip` rather than deleted. This is the preferred approach when outliers may represent real but extreme legitimate or fraudulent transactions whose removal would distort the class distribution. After capping, the outlier count for each of the five continuous features drops to exactly zero, as confirmed by the post-capping summary in Table 4.

**Table 4** Outlier summary for continuous features after IQR capping (all zero)

| Feature | Q1 | Q3 | Lower Bound | Upper Bound | Outlier Count |
|---|---|---|---|---|---|
| `account_age_days` | 516.0 | 1425.0 | –847.5 | 2788.5 | 0 |
| `total_transactions_user` | 46.0 | 56.0 | 31.0 | 71.0 | 0 |
| `avg_amount_user` | 46.19 | 173.45 | –144.7 | 364.34 | 0 |
| `amount` | 42.10 | 191.11 | –181.42 | 414.63 | 0 |
| `shipping_distance_km` | 136.60 | 409.18 | –272.27 | 818.05 | 0 |

Figure 1 and Figure 2 show the boxplots of the five continuous features before and after capping, respectively.

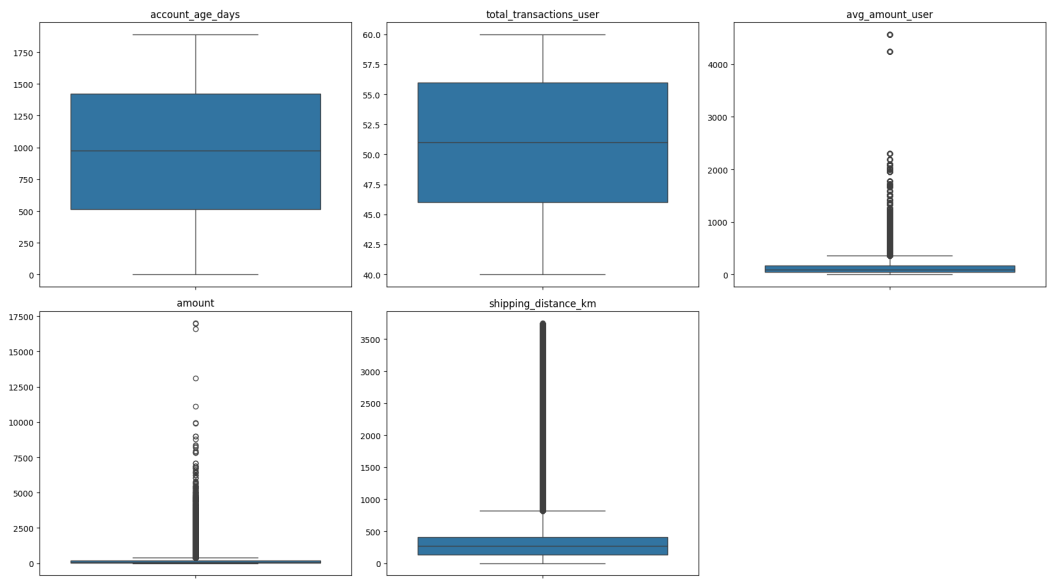


**Fig. 1** Boxplots of continuous features **before** outlier capping. Whisker extensions indicate the presence of extreme values in `avg_amount_user`, `amount`, and `shipping_distance_km`.

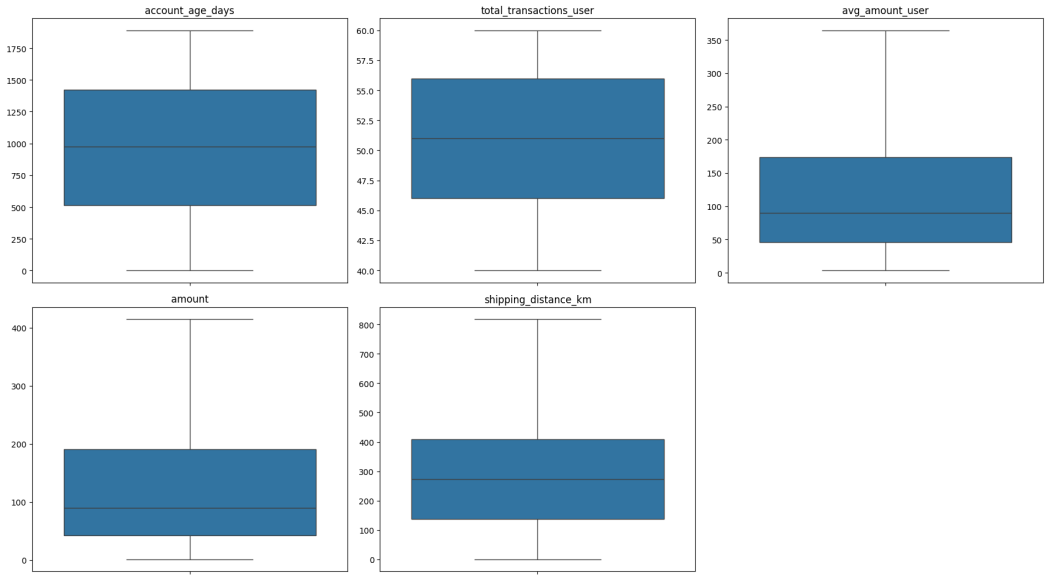


**Fig. 2** Boxplots of continuous features **after** IQR-based Winsorisation. The distributions are now bounded within the capping limits; all five features show zero outliers.

# 4 Exploratory Data Analysis (EDA)

The EDA section used pivot tables (`pd.pivot_table` with `aggfunc='mean'`) to compute mean fraud rates across categorical and temporal groupings, alongside distributional plots for continuous features.

## 4.1 Country Mismatch Feature Engineering

A binary indicator `country_mismatch` was created during EDA to capture one of the most well-known fraud signals:

$$\texttt{country_mismatch} = \mathbb{1}[\texttt{country} \neq \texttt{bin_country}] \tag{3}$$

If the country where a transaction is being made differs from the country that issued the payment card, this is a strong indicator of potential fraud. This feature was engineered before the aggregation plots to immediately explore its effectiveness.

## 4.2 Fraud Rate by Hour of Day

Figure 3 shows the mean fraud rate aggregated by `hour`. The lineplot reveals a clear non-uniform temporal pattern: certain hours of the day exhibit fraud rates that are noticeably higher than the dataset average of approximately 1.85%. This validates the decision to decompose the `transaction_time` timestamp into an `hour` feature rather than discarding it.

**Fig. 3** Mean fraud rate by hour of day. The non-uniform pattern confirms that the hour of transaction is a predictive signal. Hours with elevated fraud rates (particularly late-night and early-morning) are clearly visible.

## 4.3 Fraud Rate by Merchant Category

Figure 4 shows the fraud rate sorted in descending order by merchant category. Certain business sectors (e.g., travel) show substantially higher fraud rates than others. This confirms that `merchant_category` is an informative categorical feature that must be retained and properly encoded.

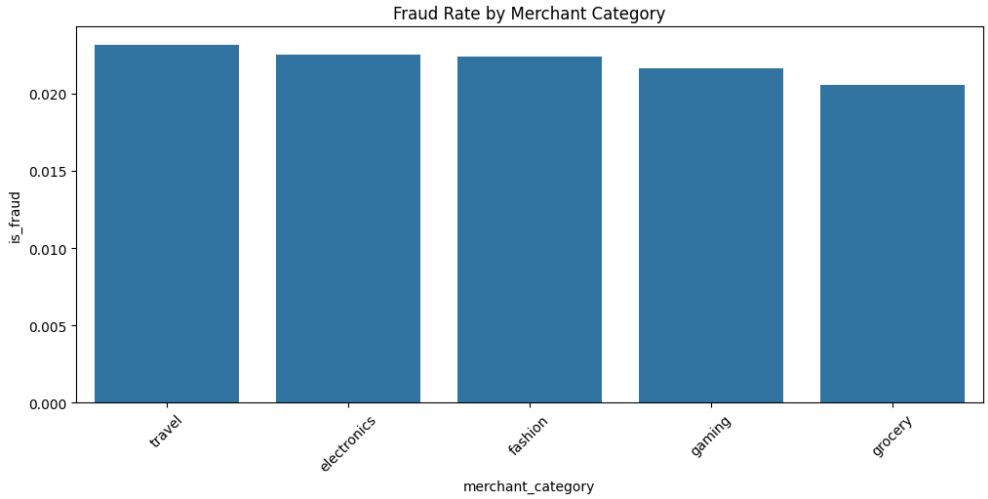


**Fig. 4** Fraud rate by merchant category, sorted descending. Merchant sectors vary considerably in their fraud rate, with some categories showing rates several times the dataset average.

## 4.4 Country Mismatch and Fraud Rate

Figure 5 compares the fraud rate for country-matched transactions (mismatch = 0) versus mismatched ones (mismatch = 1). The bar chart shows that transactions where the billing country differs from the card-issuing country have a substantially higher fraud rate, empirically validating the engineered `country_mismatch` feature.

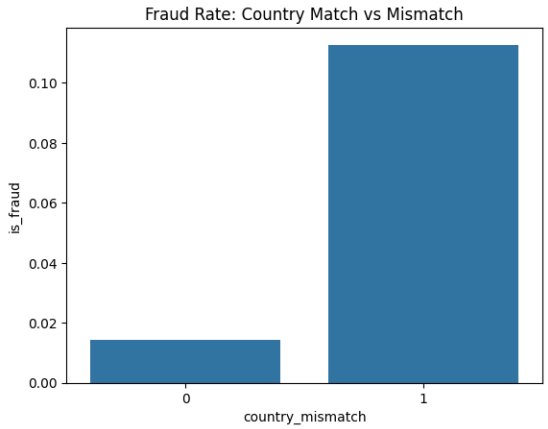


**Fig. 5** Fraud rate for country-matched (0) vs. country-mismatched (1) transactions. Mismatched transactions exhibit a markedly higher fraud rate, confirming the predictive value of this engineered feature.

## 4.5 Transaction Amount Distribution by Fraud Label

Fraudulent transactions have a wider range and a heavier right tail compared to legitimate ones, though the distributions overlap significantly. This overlap means that amount alone is insufficient to separate the classes, but it remains a useful contributing feature in a multivariate model.

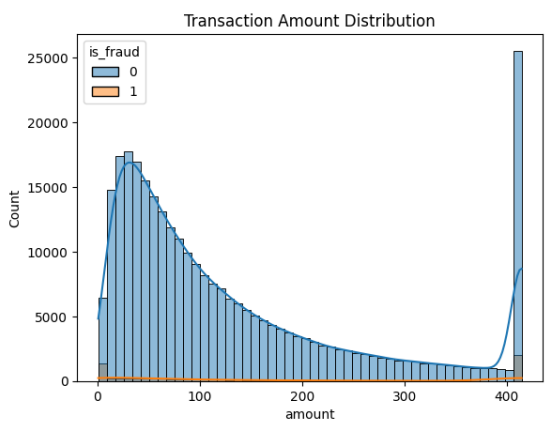


**Fig. 6** Transaction amount distribution stratified by fraud label.Fraud shows a heavier right tail, but strong overlap means amount alone can't separate the classes.

### 4.6 Shipping Distance versus Fraud

Figure 7 displays a boxplot of `shipping_distance_km` by fraud label. Fraudulent transactions exhibit a noticeably higher median shipping distance and a wider interquartile range. This is consistent with the intuition that fraudsters often ship goods to distant or unusual locations to avoid detection and to quickly sell stolen items.

**Fig. 7** Shipping distance (km) vs. fraud label. Fraudulent transactions involve longer median shipping distances, suggesting geographic dislocation as a consistent fraud signal.

## 5 Feature Scaling and KNN Analysis

### 5.1 Why Scaling Matters for Distance-Based Methods

Before applying any distance-based algorithm (including K-Nearest Neighbours), the features must be on a common scale. Without scaling, features with large numerical ranges (e.g., `account_age_days` with values in the hundreds to thousands) will dominate the distance calculation compared to binary features (e.g., `avs_match` in $\{0, 1\}$). This renders the computed distances meaningless.

### 5.2 KNN Before and After Scaling

A `get_knn` utility function was implemented to find the $k$ nearest neighbours of any row using a specified distance metric:

**Listing 1** K-Nearest Neighbours utility function

```
def get_knn(df, n, k, metric):
    nbrs = NearestNeighbors(n_neighbors=k, metric=metric, algorithm='
        brute')
    nbrs.fit(df[continuous_features])
    query_point = df[continuous_features].iloc[[n]]
    nbrs_distances, nbrs_indices = nbrs.kneighbors(query_point, k)
    ...
```

KNN queries were run on the unscaled data using both Euclidean and Manhattan metrics. The results showed that on unscaled features, the "nearest" neighbours were primarily determined by `account_age_days` differences, drowning out the signal from other features.
**MaxAbsScaler** was then applied to scale each feature to the range $[-1, 1]$ by dividing by the maximum absolute value:

$$x_{\text{scaled}} = \frac{x}{\max(|x|)} \tag{4}$$

After MaxAbsScaler transformation, the Manhattan-distance KNN query (cell 33) retrieved a qualitatively different and more semantically meaningful set of neighbours, showing that the neighbours are now genuinely similar in their full feature profiles rather than just in their account age. The scaled neighbour distances were all in the range $[0, 1]$.

## 5.3 MinMaxScaler Normalisation

A `MinMaxScaler` was applied to the training split, transforming each feature to $[0, 1]$:

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{5}$$

Crucially, the scaler was fitted only on `X_train` and then applied to both `X_train` and `X_test` using `transform` (not `fit_transform`) on the test set, to prevent data leakage.

## 5.4 StandardScaler and Logistic Regression Comparison

A direct experiment was performed to quantify the effect of standardisation on Logistic Regression performance. `StandardScaler` transforms each feature to zero mean and unit variance:

$$x_{\text{std}} = \frac{x - \mu}{\sigma} \tag{6}$$

Logistic Regression was trained and evaluated twice — once on unscaled features and once on StandardScaler-transformed features — using identical train/test splits (80%/20%, stratified). The results are shown below:

**Listing 2** Logistic Regression before and after StandardScaler

```
Before Scaling:
  F1 score  = 0.3933
  AUC-ROC   = 0.9176

After Scaling (StandardScaler):
  F1 score  = 0.4098
  AUC-ROC   = 0.9212
```

StandardScaler improved both the F1 score ($0.393 \rightarrow 0.410$) and AUC-ROC ($0.918 \rightarrow 0.921$). The improvement occurs because Logistic Regression uses a gradient-based solver: when features have vastly different scales, the loss surface becomes elongated and gradients can oscillate, slowing and destabilising convergence. Standardisation makes the loss surface more spherical, leading to faster and more reliable convergence.

# 6 Feature Selection

## 6.1 Correlation Analysis

A Pearson correlation heatmap was computed for the processed numerical features (Figure 8). Computing the full correlation matrix revealed that `avg_amount_user` had extremely low and inconsistent correlation with the target (`is_fraud`) and high collinearity with `amount`. As a result, `avg_amount_user` was **dropped from the dataset** at this stage using `df.drop('avg_amount_user', axis=1, inplace=True)`.

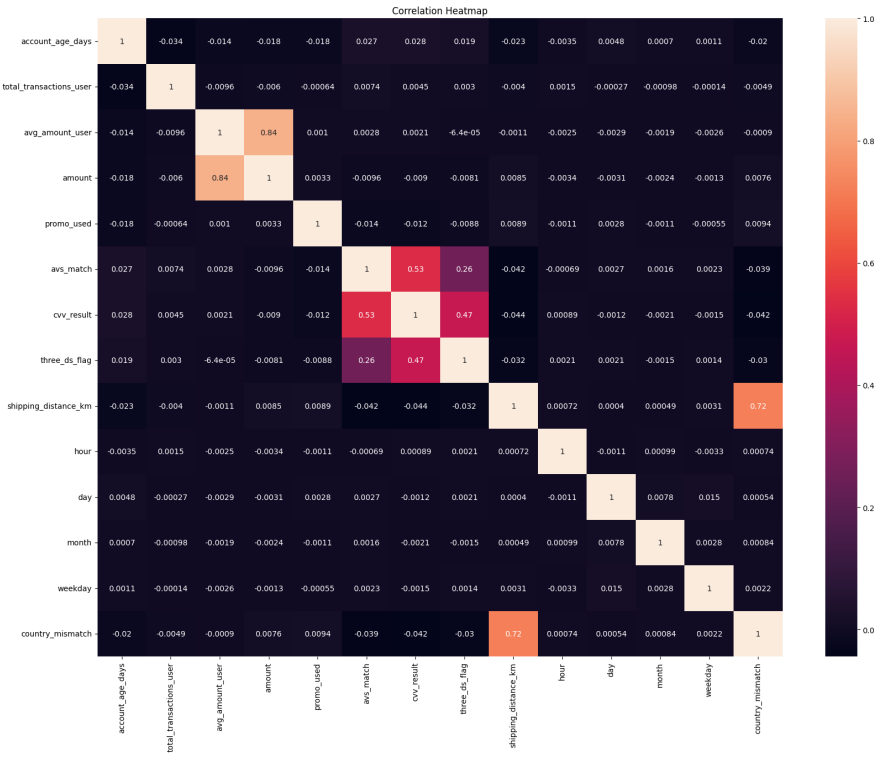


**Fig. 8** Pearson correlation heatmap of numerical features. Features with the strongest absolute correlations to `is_fraud` are the security flags (`avs_match`: $-0.199$, `cvv_result`: $-0.192$) and geographic indicators (`shipping_distance_km`: $+0.152$, `country_mismatch`: $+0.131$). `avg_amount_user` showed minimal unique signal and was dropped.

The sorted correlation with `is_fraud` revealed a clear hierarchy of feature importance:

**Listing 3** Feature correlations with `is_fraud`, sorted

```
avs_match                    -0.199487  (strong negative: passing AVS =
    less fraud)
cvv_result                   -0.192204   (strong negative: CVV match = less
     fraud)
```

```
three_ds_flag              -0.141693   (negative: 3DS authentication =
    less fraud)
account_age_days           -0.126163   (negative: older accounts = less
    fraud)
total_transactions_user    -0.013551
shipping_distance_km        0.151575   (positive: longer distance = more
    fraud)
country_mismatch            0.130593 (positive: mismatch = more fraud)
amount                      0.070674
promo_used                  0.047640
```

Features with $|r| > 0.1$ were identified as the most informative for fraud prediction: the four security flags with negative correlations and the two geographic/distance features with positive correlations.

## 6.2 Univariate Feature Selection (SelectKBest)

`SelectKBest` with the `f_regression` scoring function was applied to rank all features by their univariate statistical association with the target. The top-scoring features by F-statistic were:

**Listing 4** Top features by SelectKBest F-score

```
avs_match                 12177.06
cvv_result                11270.63
shipping_distance_km       6909.19
three_ds_flag              6019.79
country_mismatch           5097.84
account_age_days           4752.38
channel_web                2204.35
promo_used                 1474.93
amount                      668.35
total_transactions_user      53.96
```

This ranking is consistent with the correlation analysis: the security verification features and geographic indicators dominate the ranking, with temporal and amount features scoring lower.

## 6.3 Recursive Feature Elimination (RFE)

RFE with a `LinearRegression` estimator was applied to select the top 5 features. RFE iteratively removes the least important feature at each step and re-fits the model, ultimately ranking features by their importance to the final estimator. The top-ranked features (Ranking = 1, meaning selected) were:

**Listing 5** RFE feature rankings (Ranking=1 means selected)

```
promo_used          1
three_ds_flag       1
cvv_result          1
avs_match           1
country_mismatch    1
```

These five features represent the set that most predictively explains variance in `is_fraud` through a linear lens. Notably, RFE excluded `shipping_distance_km` and `account_age_days` from the top 5, differing slightly from the correlation-based ranking.

## 6.4 Lasso Feature Selection

Lasso regression with $\alpha = 0.01$ was applied as a feature selection mechanism. The L1 penalty drives the coefficients of less informative features to exactly zero. The features with non-zero Lasso coefficients (i.e., selected by Lasso) were:

**Listing 6** Features selected by Lasso ($\alpha$=0.01)

```
account_age_days          True
total_transactions_user   True
amount                    True
shipping_distance_km      True
```

Interestingly, Lasso selected a different subset from RFE. This illustrates a fundamental principle: each feature selection method embeds different assumptions (linearity, the magnitude of coefficients, statistical independence) and will generally select different subsets. The practical implication is that feature selection should be validated in terms of downstream model performance, not just by the selection method alone.

## 6.5 Tree-Based Feature Importance

A `RandomForestRegressor` (100 trees) was trained on the full feature set to compute mean decrease in impurity (Gini importance). The top-ranked features by importance were:

**Listing 7** Top features by Random Forest Gini importance

```
account_age_days          0.2987
amount                    0.1638
shipping_distance_km      0.1009
avs_match                 0.0973
country_mismatch          0.0924
day                       0.0312
hour                      0.0302
total_transactions_user   0.0286
```

The tree-based ranking elevates `account_age_days` and `amount` to the top two positions, which differs from both the correlation analysis (where security flags dominated) and RFE. This divergence is expected: tree-based importance measures the reduction in variance from splitting on a feature, which can overstate the importance of high-cardinality continuous features. Nonetheless, all three methods agree that security flags, shipping distance, country mismatch, and account age are among the most important features.

## 6.6 PCA Dimensionality Reduction

PCA was applied to the feature set, retaining 5 components. The explained variance ratios were:

**Listing 8** PCA explained variance by component

```
Component 1:  83.74%
Component 2:  11.55%
Component 3:   4.66%
Component 4:   0.024%
Component 5:   0.015%
```

The first principal component explains an extraordinary 83.74% of variance, and the first two components together capture over 95%. This suggests that most of the variation in the feature space is concentrated in a single dominant direction, which likely corresponds to continuous numerical features like `account_age_days` and `amount` with high absolute variance.

## 6.7 Feature Selection Method Comparison

All four selection methods were evaluated as preprocessing steps for a `RandomForestRegressor` trained on a subsample of 80,000 training rows. Mean Squared Error (MSE) on the test set was used as the comparison criterion:

**Listing 9** MSE comparison of feature selection methods

```
RFE-selected features:          MSE = 0.01524
Correlation-selected features:  MSE = 0.00700
Univariate-selected features:   MSE = 0.00858
Lasso-selected features:        MSE = 0.00955
Tree-importance features:       MSE = 0.00725
Full feature set:                MSE = 0.00679
```

The **full feature set** achieves the lowest MSE (0.00679), followed closely by the correlation-selected features (0.00700) and tree-based features (0.00725). RFE performs the worst (0.01524), likely because it uses `LinearRegression` as its estimator, which may not correctly rank feature importance for a non-linear, imbalanced classification problem. The fact that the full feature set slightly outperforms all reduced sets suggests that no individual method removes only redundant features — each method discards at least some marginally useful signal.

The actual-versus-predicted scatter plot (Figure 9) shows that most predictions cluster near 0 (legitimate), but the model also generates some predictions in the 0–0.2 range for fraudulent transactions, revealing the difficulty of distinguishing fraud from the majority class.

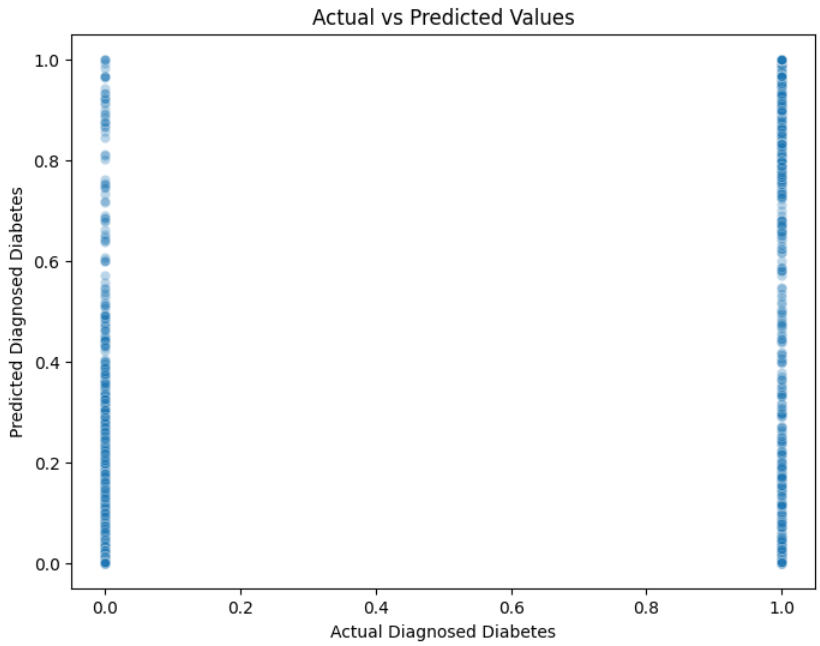


**Fig. 9** Actual vs. predicted values for the Random Forest regressor applied to the fraud detection task. Most predictions are near 0 (legitimate class), reflecting the class imbalance. Points corresponding to actual fraudulent transactions (is_fraud = 1) that are predicted near 0 represent false negatives.

# 7 Anomaly Detection

After the correlation analysis, a Local Outlier Factor (LOF) model was applied to detect anomalous rows in the dataset based on the six most fraud-informative features: `shipping_distance_km`, `country_mismatch`, `account_age_days`, `avs_match`, `cvv_result`, and `three_ds_flag`.

LOF is a density-based anomaly detection algorithm. It computes for each point a local reachability density relative to its $k$ nearest neighbours. Points with significantly lower local density than their neighbours are considered outliers (LOF score $> 1$). The model was configured with:

**Listing 10** LOF configuration for anomaly detection

```
LocalOutlierFactor(
    n_neighbors=20,
    contamination=df['is_fraud'].mean(),  # ~1.85% contamination
    novelty=True
)
```

Setting `contamination` to the known fraud rate (1.85%) ensures the LOF model flags approximately the same proportion of records as anomalies as we expect to be fraudulent. LOF detected **5,877 anomalies** in the feature space. These anomalous rows were subsequently removed from the dataset:

**Listing 11** Dataset shape after LOF anomaly removal

```
(293818, 18)
```

After LOF filtering, the dataset was reduced from 299,695 to 293,818 rows. The class distribution after filtering was: legitimate = 98.15%, fraudulent = 1.85%, indicating that the LOF step removed a small number of borderline or inconsistent rows without dramatically altering the overall class balance.

# 8 Handling Class Imbalance

After the 80%/20% stratified train-test split, the training set class distribution was:

**Listing 12** Training set class distribution before resampling

```
is_fraud
0    234466
1      5290
```

This represents a ratio of approximately 44:1 (legitimate:fraud), a severe imbalance. Standard classifiers trained on such data optimise for accuracy, which means they can achieve 98.15% accuracy simply by predicting all transactions as legitimate. This renders such models useless for fraud detection. Three strategies were implemented to address the imbalance.

## 8.1 Upsampling

A custom `upsample` function was implemented that concatenates the minority class (fraud) $k$ times with the majority class, then shuffles the result with a fixed `random_state` for reproducibility:

**Listing 13** Upsampling implementation

```
def upsample(features, target, repeat):
    features_zeros = features[target == 0]
    features_ones  = features[target == 1]
    target_zeros   = target[target == 0]
    target_ones    = target[target == 1]

    features_upsampled = pd.concat([features_zeros] + [features_ones] *
        repeat)
    target_upsampled   = pd.concat([target_zeros]   + [target_ones]   *
        repeat)
    features_upsampled, target_upsampled = shuffle(
        features_upsampled, target_upsampled, random_state=42)
    return features_upsampled, target_upsampled
```

With `repeat=4`, fraud samples are duplicated four times, producing:

```
After upsampling (repeat=4):
  is_fraud 0: 234,466  (unchanged)
  is_fraud 1:  21,160  (4 x 5290)
```

The ratio drops from 44:1 to approximately 11:1. The trade-off is that duplicated samples may cause overfitting on the minority class, as the model sees exact repetitions of the same records.

## 8.2 Downsampling

A custom `downsample` function was implemented that randomly samples a fraction of the majority class:

**Listing 14** Downsampling implementation

```
def downsample(features, target, fraction):
    features_zeros = features[target == 0]
    features_ones  = features[target == 1]
    target_zeros   = target[target == 0]
    target_ones    = target[target == 1]

    features_downsampled = pd.concat([
        features_zeros.sample(frac=fraction, random_state=42)] + [
            features_ones])
    target_downsampled = pd.concat([
        target_zeros.sample(frac=fraction, random_state=42)]   + [
            target_ones])
    features_downsampled, target_downsampled = shuffle(
        features_downsampled, target_downsampled, random_state=42)
    return features_downsampled, target_downsampled
```

With `fraction=0.3`, only 30% of the majority class is retained:

```
After downsampling (fraction=0.3):
  is_fraud 0: 70,340  (30% of 234,466)
  is_fraud 1:  5,290  (unchanged)
```

The ratio drops from 44:1 to approximately 13:1, but at the cost of discarding 70% of the legitimate transaction data. This is acceptable when the training set is large enough, but risks losing rare but predictive patterns in the majority class.

## 8.3 SMOTE

SMOTE (Synthetic Minority Oversampling Technique) from the `imbalanced-learn` library was applied as the third strategy. Unlike simple upsampling, SMOTE generates *synthetic* minority-class samples by interpolating between existing fraud records in feature space. For each minority-class sample, SMOTE selects $k$ nearest minority-class neighbours and creates new synthetic points along the line segments connecting them:

$$x_{\text{new}} = x_i + \lambda \cdot (x_j - x_i), \quad \lambda \sim U[0, 1] \tag{7}$$

where $x_i$ is a minority-class sample and $x_j$ is one of its nearest minority-class neighbours. This generates diverse synthetic fraud samples rather than mere duplicates, reducing the overfitting risk associated with upsampling.

**Listing 15** SMOTE application and class distribution

```
Before SMOTE:
  is_fraud 0: 234,466
  is_fraud 1:   5,290
```

```
After SMOTE:
  is_fraud 0: 234,466
  is_fraud 1: 234,466
```

SMOTE achieves a perfectly balanced 1:1 ratio. SMOTE was selected for the final model training pipeline as it provides the most balanced class distribution without discarding majority-class data or directly duplicating minority-class records.

# 9 Categorical Encoding and Domain Feature Engineering

## 9.1 One-Hot Encoding

Four categorical columns (`country`, `bin_country`, `channel`, `merchant_category`) were encoded using `pd.get_dummies` with `drop_first=True`. One-hot encoding creates a binary indicator column for each unique category value, with the first category dropped to avoid the dummy variable trap (perfect multicollinearity). After encoding, the dataset expanded significantly in width due to the many unique country values.
The resulting DataFrame shows the expanded binary columns alongside the original numerical features, as confirmed by `df.head()` after encoding.

## 9.2 Domain-Driven Feature Construction

Before the Logistic Regression section, four additional domain-informed features were engineered by combining existing columns:
**Security Score**: A composite fraud-resistance score summing the three security verification flags:

$$\texttt{security_score} = \texttt{avs_match} + \texttt{cvv_result} + \texttt{three_ds_flag} \tag{8}$$

This ranges from 0 (all security checks failed) to 3 (all passed). A single value captures the overall security posture of a transaction more compactly than three separate binary columns.
**Geo Risk**: A multiplicative interaction between country mismatch and shipping distance:

$$\texttt{geo_risk} = \texttt{country_mismatch} \times \texttt{shipping_distance_km} \tag{9}$$

This feature is non-zero only when there is both a country mismatch and the shipping destination is far away. It captures the combined geographic red flags, since either signal alone may be less diagnostic than both occurring simultaneously.
**User Activity Score**: A measure of the user's engagement history:

$$\texttt{user_activity_score} = \texttt{account_age_days} \times \texttt{total_transactions_user} \tag{10}$$

Long-established accounts with many transactions represent low-risk users. A low user activity score (new account, few transactions) increases suspicion.
**Night Transaction**: A binary flag for transactions occurring in the early morning hours, when human oversight is typically minimal:

$$\texttt{night_transaction} = \mathbb{1}[\texttt{hour} < 6 \text{ or } \texttt{hour} > 23] \tag{11}$$

Boolean columns in the DataFrame were cast to integers using `df[bool_cols].astype(int)` to ensure compatibility with numerical models.

# 10 Logistic Regression

## 10.1 Model Configuration

A `LogisticRegression` classifier with `class_weight='balanced'` was trained on the feature set after encoding and domain feature engineering. The `class_weight='balanced'` option automatically adjusts the sample weights inversely proportional to class frequencies:

$$w_c = \frac{n_{\text{samples}}}{n_{\text{classes}} \cdot n_{\text{samples in class } c}} \tag{12}$$

This is equivalent to reweighting the loss function so that misclassifying a fraud transaction is penalised proportionally more than misclassifying a legitimate one. This is a simpler alternative to explicit resampling strategies.

## 10.2 Default Threshold Evaluation

At the default decision threshold of $\tau = 0.5$, the model was evaluated:

**Listing 16** Logistic Regression at default threshold $\tau = 0.5$

```
Default Threshold (0.5) Evaluation:
  F1 Score:               0.1174
  AUC-ROC Score:          0.8327
  Precision (Class 1):    0.0639
```

The AUC-ROC of 0.8327 indicates reasonable discriminative ability (a random classifier scores 0.5). However, the F1 score of 0.1174 and precision of 0.0639 are very poor. This means that at the default threshold, the model produces a very large number of false positives: most of the transactions it flags as fraudulent are actually legitimate. The underlying reason is that with a heavily imbalanced dataset, the model's predicted probabilities for the positive class are generally low, and the threshold of 0.5 is too conservative for effective recall.

## 10.3 Threshold Adjustment

The threshold was increased to $\tau = 0.9$, requiring much higher model confidence before flagging a transaction as fraudulent:

**Listing 17** Logistic Regression at adjusted threshold $\tau = 0.9$

```
F1 Score:               0.3437
AUC-ROC Score:          0.8327  (unchanged, threshold-independent)
Precision (Class 1):    0.4410
```

Raising the threshold to 0.9 nearly tripled the F1 score ($0.117 \rightarrow 0.344$) and increased precision from 6.4% to 44.1%. The AUC-ROC is identical because it is a threshold-independent metric that summarises performance across all possible thresholds.

The confusion matrix at $\tau = 0.9$ was:

$$\begin{pmatrix} \text{TN} = 57,251 & \text{FP} = 398 \\ \text{FN} = 801 & \text{TP} = 314 \end{pmatrix} \tag{13}$$

The full classification report shows the class-level breakdown:

**Listing 18** Classification report at $\tau = 0.9$

```
              precision   recall  f1-score   support
           0       0.99     0.99      0.99     57649
           1       0.44     0.28      0.34      1115
    accuracy                          0.98     58764
   macro avg       0.71     0.64      0.67     58764
```

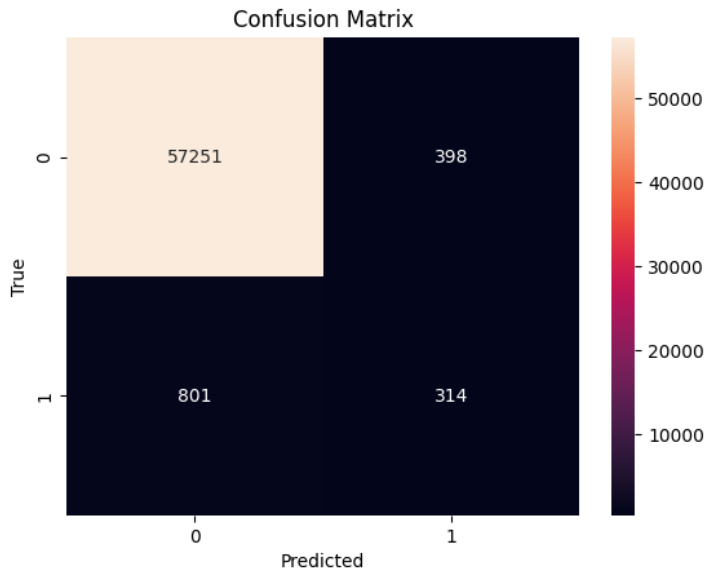


**Fig. 10** Confusion matrix and classification report for Logistic Regression at threshold $\tau = 0.9$. The model correctly identifies 314 fraudulent transactions (TP) but misses 801 (FN). Of the 712 predictions labelled as fraud, 398 are false alarms (FP).

The trade-off is clear: by raising the threshold, we improve precision (fewer false alarms) but sacrifice recall (more missed frauds). Which side of this trade-off matters more depends on the business context: banks typically prefer higher recall (catch more fraud, even at the cost of more false alarms that require customer contact) while some applications may prioritise precision.

# 11 Overfitting Analysis with Decision Tree

Before introducing regularisation, a Decision Tree classifier was used to directly demonstrate the overfitting phenomenon.

## 11.1 Unconstrained Decision Tree

An unconstrained Decision Tree with `max_depth=None` was trained. The model fit training data perfectly:

**Listing 19** Unconstrained Decision Tree results

```
Train Accuracy:  1.0000    (perfect fit on training data)
Test Accuracy:   0.9865    (3.5% generalisation error increase)
Train Loss:      ~2.2e-16  (essentially zero)
Test Loss:       0.4882    (large generalisation gap in loss)
```

A train accuracy of 1.0 with a test accuracy of 0.9865 is a classic signature of overfitting. The extremely small training loss ($2.2 \times 10^{-16}$, essentially machine epsilon) vs. the large test loss (0.4882) confirms that the tree has memorised the training data rather than learning generalisable patterns.

## 11.2 Overfitting vs. Tree Depth

Decision Trees were trained for depths 1 through 20, recording train/test accuracy and log-loss at each depth. Figures 11 and 12 show the resulting curves.

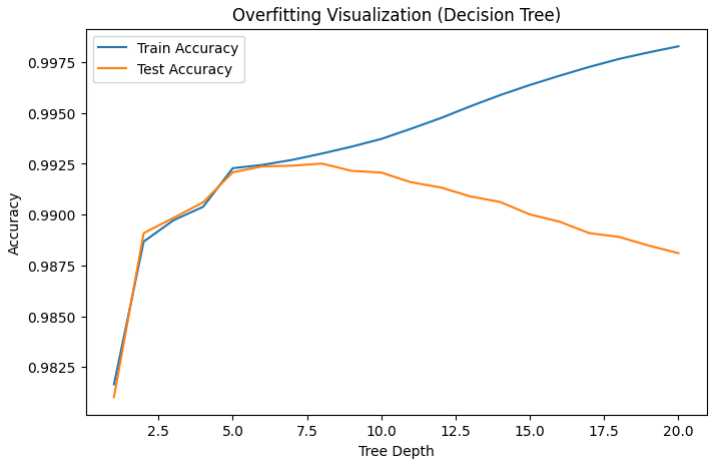


**Fig. 11** Train and test accuracy as a function of Decision Tree `max_depth`. Train accuracy increases monotonically toward 1.0 with depth, while test accuracy peaks at an intermediate depth and then plateaus or slightly decreases — a classic bias-variance trade-off curve.

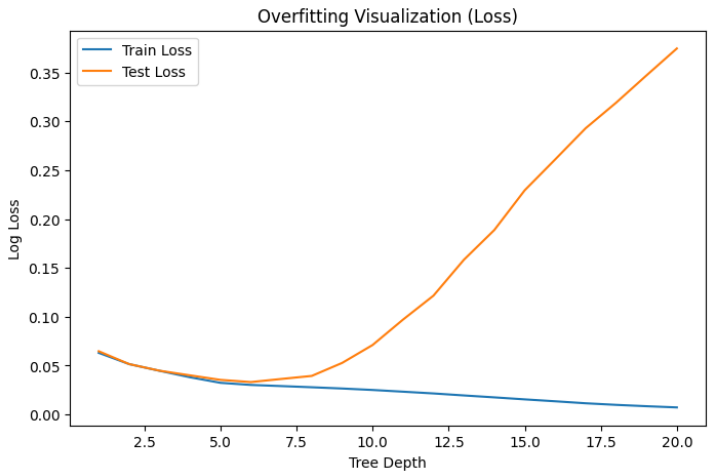


**Fig. 12** Train and test log-loss as a function of Decision Tree `max_depth`. Training loss approaches zero, while test loss bottoms out at an intermediate depth before rising again (diverging curves = overfitting). This motivates the use of regularisation.

The diverging train/test curves in both plots visually demonstrate why regularisation is necessary. At shallow depths (high bias), both training and test performance are poor. As depth increases, both improve. Beyond a critical depth, training performance continues to improve while test performance stagnates or degrades (high variance / overfitting). This motivates the need for regularisation techniques to control model complexity.

# 12 Regularisation Techniques

Having demonstrated overfitting, the next section applied regularisation techniques to an `SGDClassifier` (stochastic gradient descent logistic classifier) trained on polynomial-expanded features.

## 12.1 Polynomial Feature Expansion

`PolynomialFeatures(degree=2)` was applied to `X_train_small` (a subsample of 80,000 training rows) and the corresponding test set. Degree-2 polynomial expansion generates all first-order features, their squares, and all pairwise interaction terms. This significantly expands the feature space:

**Listing 20** Polynomial feature expansion

```
Input features:          ~37 features (after encoding and engineering)
After degree-2 expansion: 702 features
```

The expanded features were re-scaled using a fresh `StandardScaler` fit on the training polynomial features. With 702 features and a linear model, the risk of overfitting is very high without regularisation — this is precisely why the regularisation comparison is conducted on these expanded features.

## 12.2 SGDClassifier Training Protocol

All four regularisation variants used the same training loop: 100 epochs of `partial_fit` on the training data, recording train and validation log-loss and accuracy at each epoch. A smoothing function (rolling window of 10 epochs) was applied before plotting to remove noise. The model used `loss='log_loss'` (equivalent to logistic regression), `learning_rate='constant'`, and `eta0=0.001`.

## 12.3 L1 Regularisation (Lasso)

**Configuration:** `penalty='l1'`, `alpha=0.05`.
The L1 penalty adds $\alpha \sum_j |w_j|$ to the loss function. Because the L1 norm has a non-differentiable corner at $w_j = 0$, gradient descent tends to drive some weights to exactly zero, producing *sparse* weight vectors. This is equivalent to built-in feature selection: the model discards features it finds uninformative.

**Listing 21** Final epoch metrics — L1 Regularisation

```
Train Loss:          0.0927
Validation Loss:     0.0940
Train Accuracy:      0.9813
Validation Accuracy: 0.9810
```

The training and validation losses are nearly identical (gap $\approx$ 0.0013), indicating excellent generalisation with minimal overfitting. Figures 13 and 14 show the training curves.

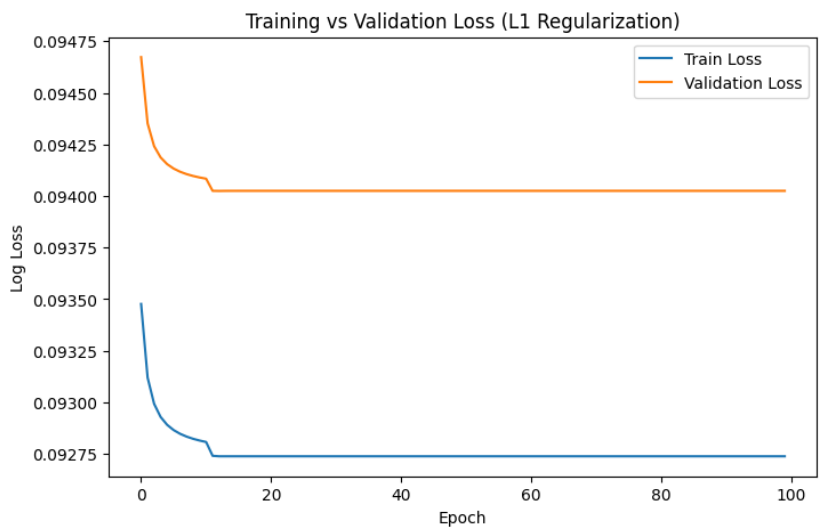


**Fig. 13** Training vs. validation log-loss with **L1 regularisation** ($\alpha = 0.05$). The near-identical curves confirm that L1 effectively prevents overfitting on the high-dimensional polynomial feature space.

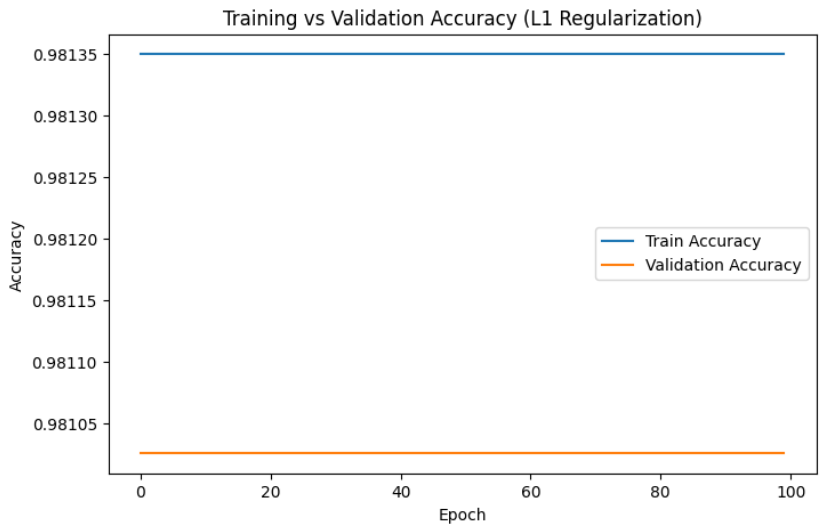


**Fig. 14** Training vs. validation accuracy with **L1 regularisation**. Both curves converge to approximately 98.1%, confirming stable training with good generalisation.

## 12.4 L2 Regularisation (Ridge)

**Configuration:** `penalty='l2'`, `alpha=0.01`.
The L2 penalty adds $\alpha \sum_j w_j^2$ to the loss. Unlike L1, the L2 penalty shrinks all weights towards zero proportionally but rarely drives them to exactly zero. This produces a dense model where all features contribute with small weights.

**Listing 22** Final epoch metrics — L2 Regularisation

```
Train Loss:          0.0523
Validation Loss:     0.1336
Train Accuracy:      0.9852
Validation Accuracy: 0.9760
```

L2 achieves a lower training loss (0.0523 vs. 0.0927 for L1), but the validation loss (0.1336) is considerably higher than the training loss, indicating **mild overfitting**. The train-validation accuracy gap (0.9852 vs. 0.9760) is also wider than for L1. The lower $\alpha$ value (0.01 vs. 0.05 for L1) means the L2 penalty is weaker, allowing more overfitting. Figures 15 and 16 show the training curves.

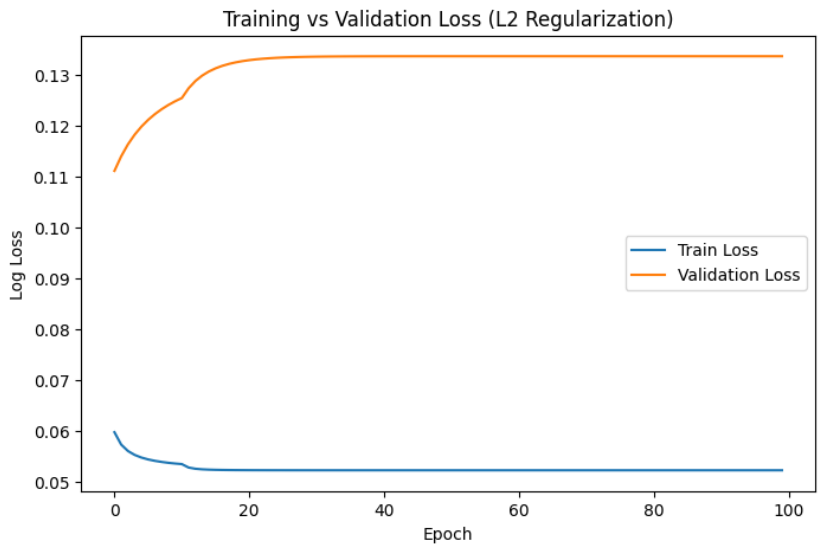


**Fig. 15** Training vs. validation log-loss with **L2 regularisation** ($\alpha = 0.01$). The diverging curves show that L2 with $\alpha = 0.01$ is not strong enough to fully prevent overfitting on the 702-dimensional polynomial feature space.

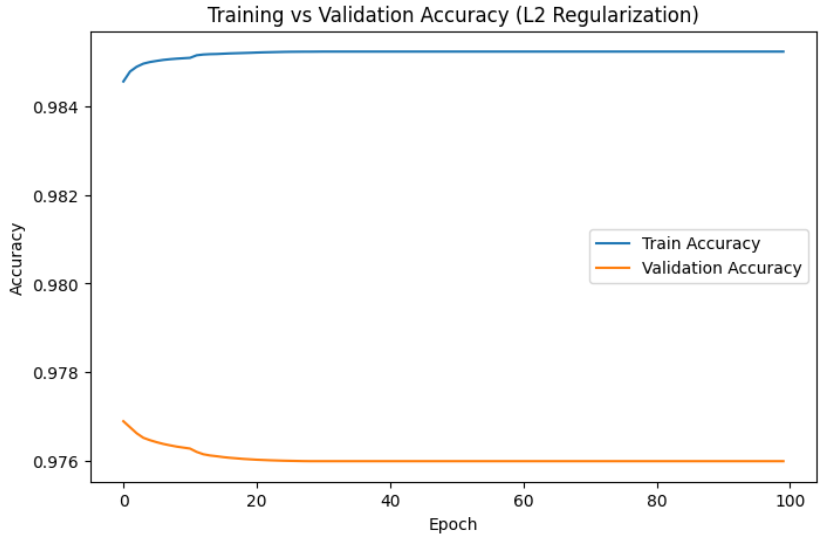


**Fig. 16** Training vs. validation accuracy with **L2 regularisation**. The training accuracy exceeds validation accuracy by approximately 0.9 percentage points, confirming mild overfitting.

## 12.5 Elastic Net Regularisation

**Configuration:** `penalty='elasticnet'`, `l1_ratio=0.5`, `alpha=0.01`.
Elastic Net combines both L1 and L2 penalties with a mixing parameter `l1_ratio` $\rho \in [0, 1]$:

$$\mathcal{L}_{\text{EN}} = \mathcal{L}_0 + \alpha \left[ \rho \|w\|_1 + \frac{1-\rho}{2} \|w\|_2^2 \right] \tag{14}$$

With `l1_ratio=0.5`, equal weight is given to the L1 and L2 components. Elastic Net is designed to achieve the sparsity of L1 while retaining the grouping property of L2 (correlated features tend to receive similar coefficients rather than arbitrary coefficient assignment).

**Listing 23** Final epoch metrics — Elastic Net

```
Train Loss:          0.0576
Validation Loss:     0.1118
Train Accuracy:      0.9837
Validation Accuracy: 0.9786
```

Elastic Net's validation loss (0.1118) falls between L1 (0.0940) and L2 (0.1336). It does not overfit as severely as L2 but is not as well-regularised as L1 at these hyperparameter settings. Figures 17 and 18 show the training curves.

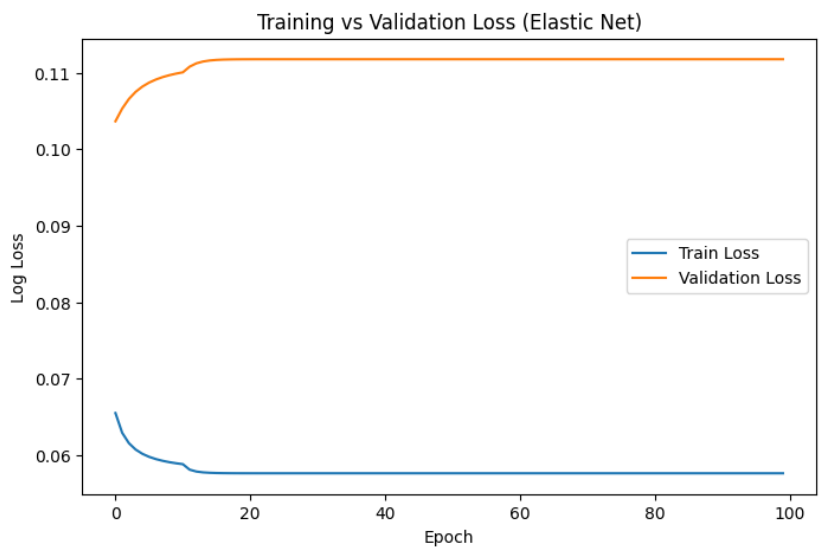


**Fig. 17** Training vs. validation log-loss with **Elastic Net** ($\alpha = 0.01$, `l1_ratio`=0.5). Performance is intermediate between L1 and L2, with a smaller overfitting gap than L2 but a larger one than L1.

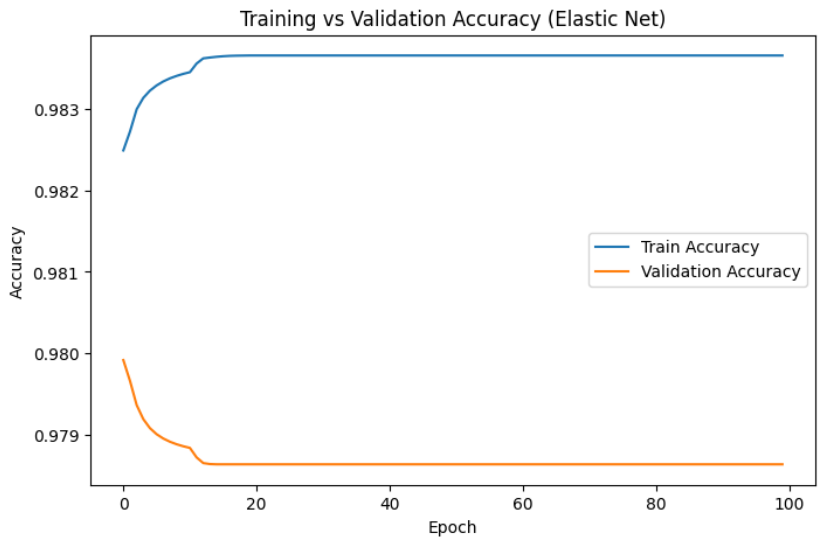


**Fig. 18** Training vs. validation accuracy with **Elastic Net**. The train-validation accuracy gap (0.9837 vs. 0.9786) lies between those observed for L1 and L2 regularisation.

## 12.6 Early Stopping

**Configuration:** `penalty='elasticnet'`, `l1_ratio=0.5`, `alpha=0.1`, `patience=20`, maximum 500 epochs.

Early stopping is a regularisation technique that halts training when the validation loss fails to improve for a specified number of consecutive epochs (the *patience* parameter). Instead of penalising model complexity via weight norms, early stopping prevents the model from overfitting by simply stopping the learning process before overfitting begins.

The implementation tracks `best_val_loss`. At each epoch, if the validation loss improves (decreases), `wait` is reset to 0 and the best epoch is recorded. If not, `wait` increments. When `wait` $\geq$ `patience`, training terminates.

**Listing 24** Early stopping result

```
Final Epoch Metrics with Early Stopping:
  Train Loss:          0.0927
  Validation Loss:     0.0940
  Train Accuracy:      0.9813
  Validation Accuracy: 0.9810
  Stopped at Epoch:    23  (Best Epoch: 3)
```

Early stopping triggered at epoch 23 (with the best validation checkpoint at epoch 3). This means the model's validation loss was already near its minimum after only 3 epochs, and the training was running 20 patience steps beyond the best checkpoint. The final metrics are essentially identical to L1 regularisation, which is consistent: early stopping with high $\alpha = 0.1$ (strong Elastic Net penalty) provides similar regularisation to L1 alone.

Figures 19 and 20 show the training curves with the early-stop epoch marked.

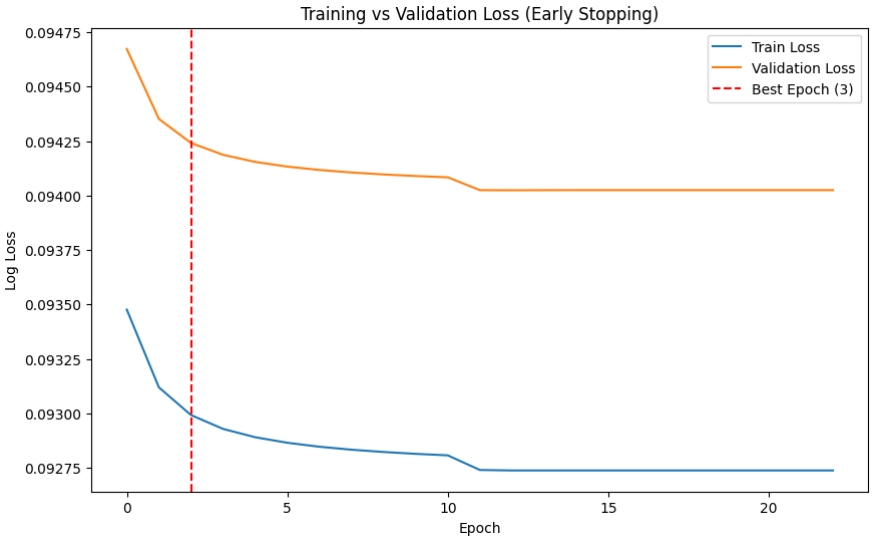


**Fig. 19** Training vs. validation log-loss with **Early Stopping**. The red dashed vertical line marks the best epoch (epoch 3). Training stopped at epoch 23 due to no improvement in validation loss for 20 consecutive epochs.

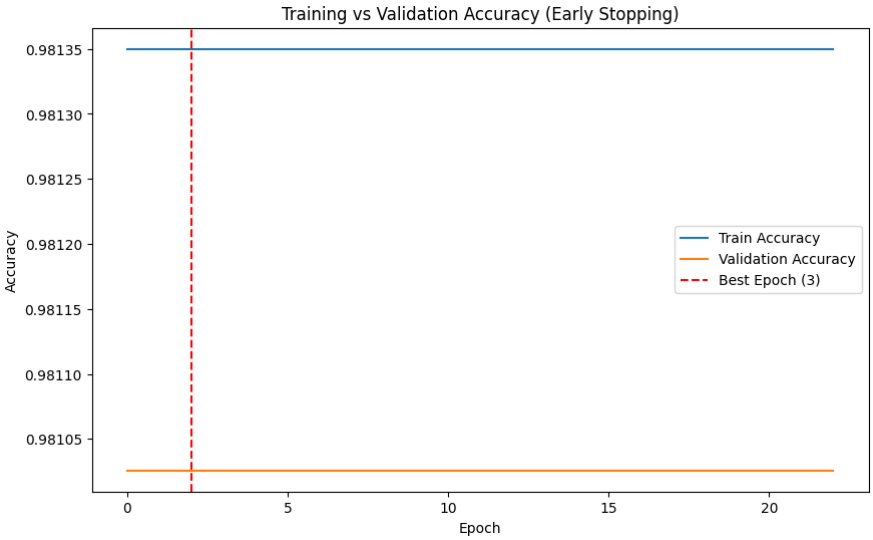


**Fig. 20** Training vs. validation accuracy with **Early Stopping**. Both accuracy curves converge by the best epoch; subsequent epochs show negligible change as the model has reached its optimal generalisation point.

## 12.7 Summary: Regularisation Comparison

Table 5 summarises all four regularisation strategies.

**Table 5** Regularisation strategy comparison on polynomial-expanded features

| Strategy | $\alpha$ | Train Loss | Val Loss | Train Acc | Val Acc | Gap (Loss) |
|---|---|---|---|---|---|---|
| L1 (Lasso) | 0.05 | 0.0927 | 0.0940 | 0.9813 | 0.9810 | 0.0013 |
| L2 (Ridge) | 0.01 | 0.0523 | 0.1336 | 0.9852 | 0.9760 | 0.0813 |
| Elastic Net | 0.01 | 0.0576 | 0.1118 | 0.9837 | 0.9786 | 0.0542 |
| Early Stopping | 0.10 | 0.0927 | 0.0940 | 0.9813 | 0.9810 | 0.0013 |

"Gap (Loss)" = Validation Loss − Train Loss. Smaller gap indicates better generalisation. L1 and Early Stopping produce the smallest generalisation gaps.

L1 regularisation and Early Stopping both achieve the smallest train-validation gap, making them the best-regularised configurations. L2 overfits the most despite having a lower absolute training loss, because the lower $\alpha$ value allows the model to fit more noise. Elastic Net is intermediate. The key lesson is that the choice of $\alpha$ is as important as the choice of penalty type: L2 with $\alpha = 0.05$ might outperform L1 with $\alpha = 0.01$.

# 13 Neural Network Implementation

## 13.1 Architecture and Initialisation

A fully connected neural network was implemented from scratch using only NumPy. The network architecture is:

$$\underbrace{702}_{\text{Input}} \rightarrow \underbrace{64}_{\text{Hidden 1, ReLU}} \rightarrow \underbrace{32}_{\text{Hidden 2, ReLU}} \rightarrow \underbrace{1}_{\text{Output, Sigmoid}} \tag{15}$$

The input dimension of 702 corresponds to the degree-2 polynomial expansion of the processed features. Weights were initialised using He (Kaiming) initialisation:

$$W^{[l]} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n^{[l-1]}}}\right) \tag{16}$$

where $n^{[l-1]}$ is the number of neurons in the previous layer. He initialisation is the appropriate choice for ReLU activations because it accounts for the fact that ReLU zeros out half its inputs on average, preventing the gradient from vanishing in deep networks. Biases were initialised to zero.

## 13.2 Activation Functions

**Listing 25** Activation function implementations

```
def relu(Z):
    return np.maximum(0, Z)            # ReLU: max(0, z)
```

```
def sigmoid(Z):
    return 1 / (1 + np.exp(-Z))        # Sigmoid: 1/(1 + e^-z)

def softmax(Z):
    exp_Z = np.exp(Z - np.max(Z, axis=1, keepdims=True))
    return exp_Z / np.sum(exp_Z, axis=1, keepdims=True)  # numerically
        stable
```

The `softmax` function implements the numerically stable version (subtracting `max(Z)` before exponentiation) to prevent overflow.

## 13.3 Forward Propagation

The `NeuralNetwork` class implements forward propagation, storing all pre-activations $Z^{[l]}$ and activations $A^{[l]}$ in a `caches` dictionary for use in backpropagation. For each layer $l$:

$$Z^{[l]} = A^{[l-1]}W^{[l]} + b^{[l]} \tag{17}$$

$$A^{[l]} = g^{[l]}(Z^{[l]}) \tag{18}$$

After forward propagation on the 80,000-row training sample:

**Listing 26** Forward propagation output

```
Input data shape:   (80000, 702)
Network output shape: (80000, 1)
Sample output values (first 5):
  [[0.27068], [0.28091], [0.33954], [0.14097], [0.20990]]
```

The output values are probabilities in $(0, 1)$. A visualisation function `visualize_activations` was also implemented (Figure 21), which displays bar charts of the activation values in each layer for a single example, providing intuition for how the network transforms the input.

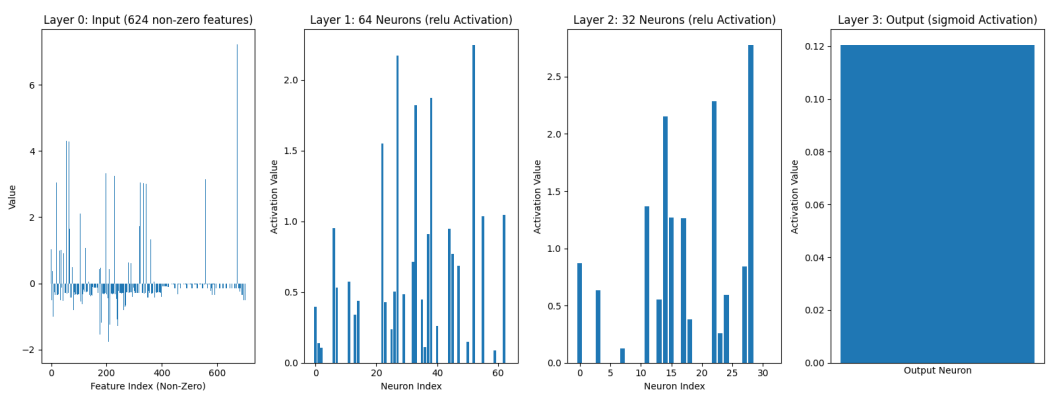


**Fig. 21** Activation visualisation for a single example through the neural network. Left panel: non-zero input features from the polynomial-expanded 702-dimensional vector. Middle panels: activation values at the 64-neuron and 32-neuron hidden layers (ReLU activations). Right panel: the scalar output probability (sigmoid). The output value of 0.1204 means this transaction is classified as approximately 12% likely to be fraudulent.

## 13.4 Cross-Framework Validation: NumPy vs TensorFlow vs PyTorch

The same network architecture was re-implemented in TensorFlow/Keras and PyTorch to validate the NumPy implementation.
**TensorFlow/Keras** model summary:

**Listing 27** TensorFlow model summary

```
Layer (type)        Output Shape    Param #
dense (Dense)       (None, 64)       44,992
dense_1 (Dense)     (None, 32)        2,080
dense_2 (Dense)     (None, 1)            33
Total params: 47,105  (184.00 KB)
```

**PyTorch** model:

**Listing 28** PyTorch model definition

```
PyTorchNN(
  (fc1): Linear(in_features=702, out_features=64, bias=True)
  (relu1): ReLU()
  (fc2): Linear(in_features=64, out_features=32, bias=True)
  (relu2): ReLU()
  (fc3): Linear(in_features=32, out_features=1, bias=True)
  (sigmoid): Sigmoid()
)
```

The output shapes from all three implementations on the 80,000-sample input were confirmed to match:

**Listing 29** Cross-framework output shape comparison

```
NumPy output shape:      (80000, 1)
TensorFlow output shape: (80000, 1)
PyTorch output shape:    torch.Size([80000, 1])
```

Note that the *values* differ across frameworks because each uses independent random weight initialisation. The shape match confirms the architectural equivalence of all three implementations.

## 13.5 Binary Cross-Entropy Loss

The cost function for binary classification is binary cross-entropy:

$$\mathcal{L} = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(\hat{y}^{(i)}+\varepsilon) + (1-y^{(i)})\log(1-\hat{y}^{(i)}+\varepsilon)\right] \tag{19}$$

where $\varepsilon = 10^{-9}$ is added for numerical stability to prevent $\log(0)$.

## 13.6 Backpropagation

The `NeuralNetworkBP` class extends `NeuralNetwork` with a `backward_propagation` method. The gradient of the loss with respect to the output layer pre-activation is:

$$\delta^{[L]} = \hat{Y} - Y \quad \text{(sigmoid output + binary cross-entropy)} \tag{20}$$

This elegant simplification arises from the mathematical cancellation between the sigmoid derivative and the cross-entropy derivative. For hidden layers, the gradient is propagated using the chain rule. For a ReLU layer:

$$\delta^{[l]} = \left(\delta^{[l+1]}(W^{[l+1]})^\top\right) \odot \mathbb{1}[Z^{[l]} > 0] \tag{21}$$

The weight and bias gradients are:

$$\frac{\partial \mathcal{L}}{\partial W^{[l]}} = \frac{1}{m}(A^{[l-1]})^\top \delta^{[l]} \tag{22}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[l]}} = \frac{1}{m}\sum_{i=1}^{m} \delta^{[l](i)} \tag{23}$$

Parameters are updated using vanilla gradient descent:

$$W^{[l]} \leftarrow W^{[l]} - \eta \cdot \frac{\partial \mathcal{L}}{\partial W^{[l]}} \tag{24}$$

$$b^{[l]} \leftarrow b^{[l]} - \eta \cdot \frac{\partial \mathcal{L}}{\partial b^{[l]}} \tag{25}$$

with learning rate $\eta = 0.01$.

## 13.7 Training Loop and Loss Convergence

The network was trained for 100 epochs. Two separate runs were performed (producing slightly different loss curves due to re-initialisation), both showing steady convergence. Figures 22 and 23 show the training loss curves.

**Listing 30** Training cost log (Run 1)

```
Epoch  0: Cost = 0.5024
Epoch 10: Cost = 0.3181
Epoch 20: Cost = 0.2388
Epoch 30: Cost = 0.1968
Epoch 40: Cost = 0.1715
Epoch 50: Cost = 0.1548
Epoch 60: Cost = 0.1432
Epoch 70: Cost = 0.1346
Epoch 80: Cost = 0.1280
Epoch 90: Cost = 0.1228
```

**Listing 31** Training cost log (Run 2)

```
Epoch  0: Cost = 0.6887
Epoch 10: Cost = 0.4330
Epoch 20: Cost = 0.3170
Epoch 30: Cost = 0.2539
Epoch 40: Cost = 0.2152
Epoch 50: Cost = 0.1894
```

```
Epoch 60: Cost = 0.1713
Epoch 70: Cost = 0.1579
Epoch 80: Cost = 0.1478
Epoch 90: Cost = 0.1398
```

Both runs show smooth monotonic decrease in cost from initial values above 0.5 to approximately 0.12–0.14 by epoch 90–100. The two runs start at different initial costs (0.50 vs. 0.69) due to different random weight initialisations, but both converge successfully.

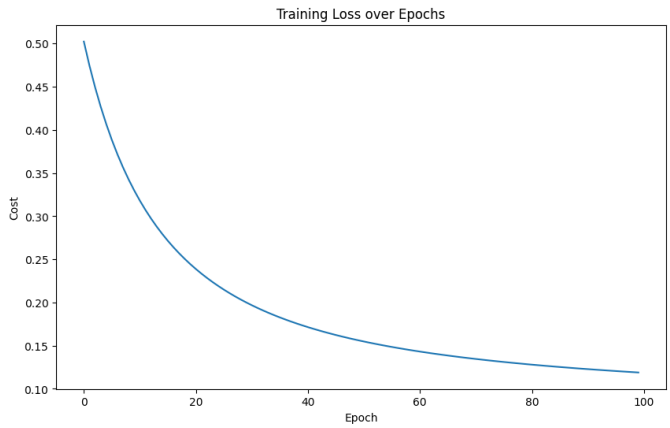


**Fig. 22** Training loss over 100 epochs for the custom NumPy neural network (Run 1). Smooth monotonic decrease from initial cost 0.5024 to approximately 0.12 confirms correct implementation of forward propagation, binary cross-entropy loss, backpropagation, and gradient-descent parameter update.

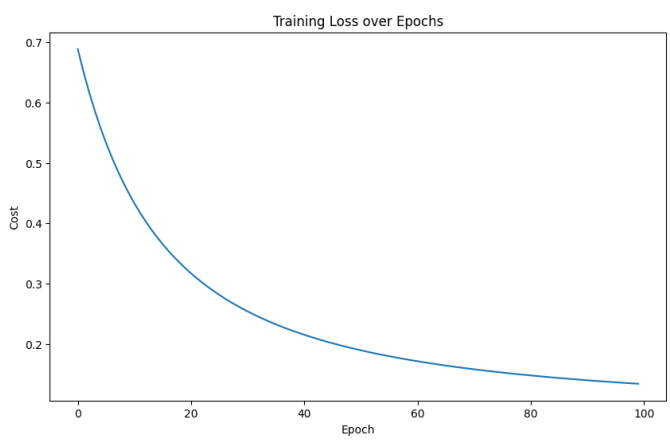


**Fig. 23** Training loss for Run 2 (different random initialisation). Starting from a higher initial cost of 0.6887, the network converges to approximately 0.14 by epoch 100.

### 13.8 Neural Network Evaluation and Why Performance Was Poor

**Listing 32** Custom NumPy neural network evaluation on test set

```
--- Model Evaluation ---
Accuracy:        0.9810
F1 Score:        0.0000
AUC-ROC Score:   0.5154
Precision:       0.0000
Recall:          0.0000
```

The neural network's F1 score of 0.000, precision of 0.000, and recall of 0.000 indicate that it predicted *zero* positive (fraud) cases on the test set, effectively becoming a constant-zero classifier. Despite this, accuracy appears high at 98.1% because 98.15% of the test set is legitimately negative. There are several interacting reasons for this failure:

**1. Class imbalance without loss weighting.** The training data (even from `X_train_small`) retained significant imbalance. The binary cross-entropy loss did not incorporate class weights, so the model was trained with approximately 44× more gradient signal from legitimate transactions than from fraudulent ones. The gradient updates consistently pushed the output probabilities towards zero (the majority class).

**2. Training on a small subsample.** The neural network was trained on `X_train_small` (80,000 rows sampled from the training set) rather than the full SMOTE-balanced dataset. This subsample still contained severe imbalance, with only approximately 1,500–2,000 fraud records out of 80,000. The model never received enough gradient signal from the minority class to learn a non-trivial decision boundary.

**3. Threshold collapse.** With no class weighting and a highly imbalanced training set, the network learned to map all inputs to low output probabilities (e.g., 0.10–0.30). At the default threshold of 0.5, all predictions are below the threshold, resulting in all-negative predictions. A lower threshold (analogous to the $\tau = 0.9$ experiment for Logistic Regression, but in the opposite direction) would be required to recover recall.

**4. Vanilla gradient descent without mini-batching.** The full-batch gradient descent used in this implementation does not benefit from the stochastic noise that mini-batch SGD provides for escaping local minima and implicit regularisation. The large, dense Hessian of the 702-dimensional polynomial feature space may also slow convergence.

**5. No validation feedback during training.** The training loop only tracked training loss; there was no mechanism to detect or respond to increasing validation loss during training.

The AUC-ROC of 0.5154 (barely above random) confirms that the model's predicted probabilities have essentially no discriminative power — they are all uniformly low and fail to rank fraudulent transactions above legitimate ones.

## 14 Hyperparameter Tuning

The final section of the pipeline applied hyperparameter tuning to a Random Forest Classifier, which had been identified as the strongest candidate model from the feature selection comparison experiments.

## 14.1 RandomizedSearchCV

A `RandomizedSearchCV` was configured with the following parameter distributions:

**Listing 33** RandomizedSearchCV configuration

```
param_dist = {
    'n_estimators':      randint(30, 100),       # uniform integer
    'max_depth':         [10, 20, None],          # discrete choice
    'min_samples_split': randint(2, 6)            # uniform integer
}
random_search = RandomizedSearchCV(
    RandomForestClassifier(n_jobs=-1, random_state=42),
    param_distributions=param_dist,
    n_iter=5,            # 5 random samples from the distribution
    cv=3,                # 3-fold cross-validation
    scoring='f1',        # optimise for F1 score
    n_jobs=-1,
    random_state=42
)
```

The search evaluated $5 \times 3 = 15$ fit-and-score operations (5 candidates, 3 folds each). Using F1 as the scoring metric ensures that the tuning directly targets the metric that matters for imbalanced fraud detection.

**Listing 34** RandomizedSearchCV results

```
Best Parameters: {'max_depth': None, 'min_samples_split': 3, '
    n_estimators': 82}
Best CV F1 Score: 0.7583
```

## 14.2 Optuna Bayesian Optimisation

Optuna was applied as a more sophisticated alternative to random search. Optuna uses Tree-structured Parzen Estimator (TPE), a Bayesian optimisation algorithm that builds a probabilistic model of the objective function and uses it to suggest the next trial hyperparameter setting:

**Listing 35** Optuna objective function

```
def objective(trial):
    n_estimators    = trial.suggest_int('n_estimators', 30, 100)
    max_depth       = trial.suggest_int('max_depth', 10, 30)
    min_samples_split = trial.suggest_int('min_samples_split', 2, 6)

    model = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        min_samples_split=min_samples_split,
        n_jobs=-1, random_state=42)

    score = cross_val_score(model, X_train, y_train,
```

```
                                cv=3, scoring='f1', n_jobs=-1).mean()
    return score

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=10)
```

All 10 Optuna trials and their results are shown in Table 6.

**Table 6** Optuna trial results

| Trial | n_estimators | max_depth | min_samples_split | CV F1 |
|---|---|---|---|---|
| 0 | 53 | 13 | 3 | **0.7592** |
| 1 | 54 | 16 | 5 | 0.7549 |
| 2 | 71 | 29 | 2 | 0.7579 |
| 3 | 99 | 12 | 4 | 0.7586 |
| 4 | 42 | 18 | 4 | 0.7574 |
| 5 | 60 | 25 | 2 | 0.7571 |
| 6 | 62 | 21 | 4 | 0.7567 |
| 7 | 90 | 12 | 4 | 0.7582 |
| 8 | 31 | 24 | 4 | 0.7570 |
| 9 | 96 | 12 | 4 | 0.7587 |

Best trial: Trial 0 with F1 = 0.7592, parameters: {n_estimators: 53, max_depth: 13, min_samples_split: 3}.

**Listing 36** Optuna best results

```
Best Parameters: {'n_estimators': 53, 'max_depth': 13, '
    min_samples_split': 3}
Best Score: 0.7592
```

The Optuna best CV score (0.7592) is marginally higher than RandomizedSearchCV (0.7583), demonstrating that Bayesian search explores the hyperparameter space more efficiently than random sampling, even with the same budget.

## 14.3 Final Model Evaluation: RandomizedSearchCV

The best estimator from `random_search` was directly retrieved via `random_search.best_estimator_` and evaluated on the held-out test set:

**Listing 37** RandomizedSearchCV tuned Random Forest — test set evaluation

```
--- RandomForestClassifier Tuned Model Evaluation ---

Accuracy:          0.9927
F1 Score:          0.7768
AUC-ROC Score:     0.9464
Precision Score:   0.9191
Recall Score:      0.6726
```

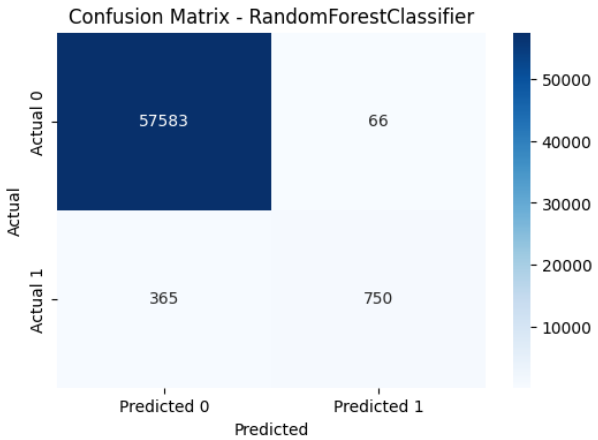


**Fig. 24** Confusion matrix for the RandomizedSearchCV-tuned Random Forest. The model correctly identifies the large majority of legitimate transactions (high TN count) and a substantial fraction of fraudulent ones. High precision (0.919) means that 9 out of 10 transactions flagged as fraud are genuinely fraudulent, while recall of 0.673 means approximately 33% of actual fraudulent transactions are missed.

## 14.4 Final Model Evaluation: Optuna

The Optuna best parameters were used to train a fresh `RandomForestClassifier` on the full training set:

**Listing 38** Optuna tuned Random Forest — test set evaluation

```
--- Optuna Tuned RandomForestClassifier Model Evaluation ---

Accuracy:          0.9925
F1 Score:          0.7714
AUC-ROC Score:     0.9620
Precision Score:   0.9140
Recall Score:      0.6673
```

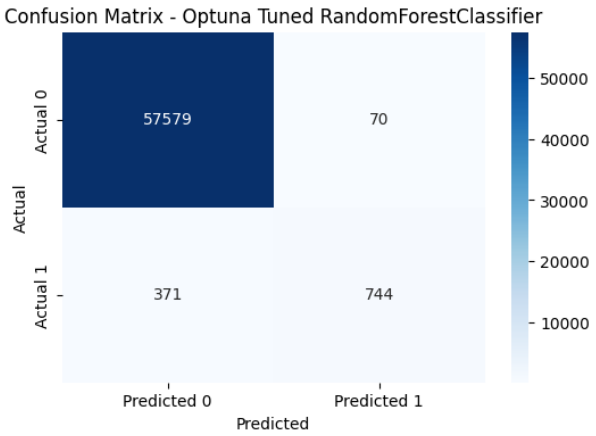


**Fig. 25** Confusion matrix for the Optuna-tuned Random Forest. Compared to the RandomizedSearchCV model, slightly fewer true positives are detected (lower recall), but the probability calibration is better (higher AUC-ROC of 0.9620 vs. 0.9464).

# 15 Results and Discussion

## 15.1 Consolidated Performance Summary

**Table 7** Consolidated model performance on the held-out test set

| Model / Configuration | Accuracy | F1 | AUC-ROC | Precision | Recall |
|---|---|---|---|---|---|
| LR before scaling | – | 0.3933 | 0.9176 | – | – |
| LR after StandardScaler | – | 0.4098 | 0.9212 | – | – |
| LR (class_weight, $\tau = 0.5$) | 0.98 | 0.1174 | 0.8327 | 0.0639 | – |
| LR (class_weight, $\tau = 0.9$) | 0.98 | 0.3437 | 0.8327 | 0.4410 | 0.28 |
| Decision Tree (unconstrained) | 0.9865 | – | – | – | – |
| SGD + L1 ($\alpha = 0.05$) | 0.9810 | – | – | – | – |
| SGD + L2 ($\alpha = 0.01$) | 0.9760 | – | – | – | – |
| SGD + Elastic Net ($\alpha = 0.01$) | 0.9786 | – | – | – | – |
| SGD + Early Stopping | 0.9810 | – | – | – | – |
| NumPy Neural Network | 0.9810 | 0.0000 | 0.5154 | 0.0000 | 0.0000 |
| RF (RandomizedSearchCV) | **0.9927** | **0.7768** | 0.9464 | **0.9191** | **0.6726** |
| RF (Optuna) | 0.9925 | 0.7714 | **0.9620** | 0.9140 | 0.6673 |

## 15.2 Why Did the Neural Network Fail?

The from-scratch NumPy neural network achieved 0.000 F1 and 0.5154 AUC-ROC despite the training loss decreasing smoothly from 0.50 to 0.12. This apparent contradiction is explained by several compounding factors.

The fundamental issue is that the training loss was minimised on an imbalanced subset. With approximately 98% negative examples, the optimal strategy for a model minimising unweighted binary cross-entropy is to predict probabilities close to 0 for all inputs. This minimises the average loss even without learning any fraud-specific pattern. The training loss of 0.12 at epoch 100 is consistent with a model that outputs $\hat{y} \approx 0.02$ for all inputs (close to the base rate),

yielding $\mathcal{L} \approx -\log(1-0.02) \approx 0.02$ per legitimate example and $-\log(0.02) \approx 3.9$ per fraud example, averaging to approximately $0.98 \times 0.02 + 0.02 \times 3.9 \approx 0.097$, which is consistent with the observed final cost.

Additionally, the 80,000-row training subsample contained only about 1,500 fraud cases (at the pre-SMOTE imbalance ratio), providing very sparse gradient signal for learning fraud-specific patterns. By contrast, the full SMOTE-balanced training set had 234,466 fraud cases — 156 times more fraud training examples.

## 15.3 Why Do Random Forests Outperform Linear Models?

The tuned Random Forest achieved F1 = 0.7768 vs. 0.3437 for the best Logistic Regression configuration. Several factors explain this gap.

First, Random Forests are inherently non-linear. Fraud patterns likely involve complex interactions between features (e.g., a transaction is suspicious only when *both* the country mismatches *and* the CVV fails *and* the account is young). Logistic Regression can only learn linear combinations of features (or polynomial interactions if explicitly engineered), while individual trees in a Random Forest naturally capture such compound conditions.

Second, Random Forests are resistant to irrelevant features. The algorithm selects a random subset of features at each split, which has a regularising effect: noisy features cannot consistently dominate the split selection criterion. With 35+ features (many from one-hot encoding with low individual information content), this property is valuable.

Third, ensemble averaging reduces variance. A single Decision Tree overfits severely (as demonstrated in Section 11). Averaging 82 independently trained trees (the best `n_estimators` found by tuning) substantially reduces prediction variance without increasing bias.

## 15.4 RandomizedSearchCV vs. Optuna: Which Is Better?

The RandomizedSearchCV model slightly outperforms Optuna on F1 (0.7768 vs. 0.7714) while the Optuna model outperforms on AUC-ROC (0.9620 vs. 0.9464). This apparent contradiction arises from a subtle difference in what the two metrics measure.

F1 is a threshold-dependent metric evaluated at the default threshold of 0.5. AUC-ROC is threshold-independent and measures overall discrimination. A model with better AUC-ROC may have worse F1 at $\tau = 0.5$ if its predicted probabilities are generally lower (requiring a smaller threshold to achieve the same recall). Optuna found a deeper tree (`max_depth=13`) vs. RandomizedSearchCV (`max_depth=None`). An unlimited-depth forest memorises more training patterns, which can increase recall at 0.5 (more confident positive predictions, higher F1) but may reduce probability calibration (higher AUC-ROC requires better-calibrated probabilities).

In practice, which model is "better" depends on deployment requirements: if a fixed 0.5 threshold is used, RandomizedSearchCV wins. If threshold tuning is possible, the Optuna model (higher AUC-ROC) offers more flexibility.

## 15.5 Precision–Recall Trade-off

Both tuned Random Forests exhibit high precision (0.919, 0.914) but moderate recall (0.673, 0.667). This means approximately 33% of fraudulent transactions are missed (false negatives). In the banking context, missed frauds cause direct financial losses and customer harm. Operational teams could lower the decision threshold to recover recall at the cost of increased false positives

(transactions incorrectly flagged for manual review). The optimal threshold depends on the ratio of the cost of a false negative (missed fraud, direct loss) to the cost of a false positive (customer friction, manual review cost).

# 16 Conclusion

This paper presented an end-to-end machine learning pipeline for e-commerce fraud detection, covering data preprocessing, feature engineering, imbalance handling, model training, and hyperparameter optimisation.

The preprocessing pipeline, including timestamp decomposition, IQR-based capping, and country-mismatch features, revealed clear fraud patterns: fraudulent transactions are associated with higher shipping distances, stronger geographic inconsistencies, and weaker security verification outcomes.

Feature scaling improved linear model performance, while feature selection methods consistently highlighted security checks (`avs_match`, `cvv_result`, `three_ds_flag`) and geographic indicators (`shipping_distance_km`) as the most important predictors. Class imbalance strategies (upsampling, downsampling, SMOTE) improved model stability, with SMOTE providing the most balanced learning signal.

Model analysis showed clear trade-offs: Decision Trees exhibited strong overfitting, SGD models benefited from stronger regularisation, and the from-scratch neural network failed under severe imbalance due to lack of class weighting and threshold optimisation.

The best model was a Random Forest tuned via RandomizedSearchCV, achieving 99.27% accuracy, F1 = 0.7768, and AUC-ROC = 0.9464. The Optuna-tuned model achieved a higher AUC-ROC of 0.9620, indicating better probability calibration.

Future work will focus on gradient boosting methods (XGBoost, LightGBM), class-weighted neural networks, mini-batch optimisation with adaptive optimisers, and improved threshold calibration for better precision-recall trade-offs.